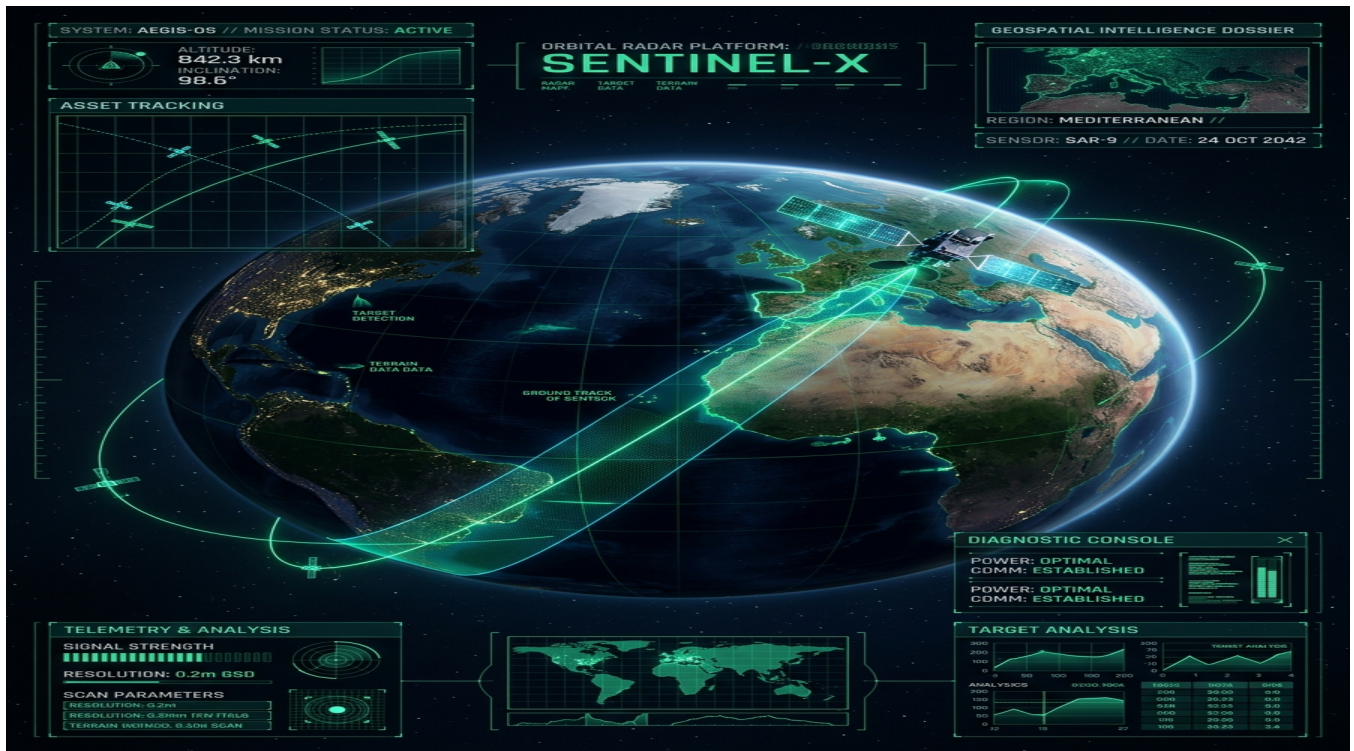




SATELLITE IMAGE THREAT DETECTION (SITP)

LOW-LEVEL DETAILED DESIGN SPECIFICATION (LLD)

Document Version: 3.0.0 | Classification: Technical Engineering Spec

Lead Module: src.SITP | Release Date: September 2026

Topics: Component Classes, ONNX Engine Logic, Tile Rejection Algorithm, Loss Formulations, Batched NMS, Latency Instrumentation, API Payloads

Key Design Principle: This document specifies low-level algorithms, class signatures, ONNX Runtime auto-select logic, background tile rejection thresholds, multi-phase latency instrumentation, mathematical coordinate transformations, neural network loss functions, high-threat brief generation, and API schemas.

1. Modular Component Specifications

A. DataIngestion Component (data_ingestion.py)

Class: DataIngestion(config: DataIngestionConfig)

Method: initiate_data_ingestion() -> tuple[pd.DataFrame, dict, Path, Path]

Logic: Loads xView_train.geojson features, extracts bounding polygon bounds [min_x, min_y, max_x, max_y], maps 60+ raw xView IDs to contiguous indices [0..61], and partitions images with a reproducible seed to guarantee zero chip overlap.

B. DataTransformation Component (data_transformation.py)

Class: DataTransformation(config: DataTransformationConfig)

Method: initiate_data_transformation(ann_df, image_split, image_dir, output_dir) -> tuple[Path, dict]

Tiling Math: Slices GeoTIFF rasters into 512x512 chips using sliding stride $S = T - O = 512 - 148 = 364$ px. Bounding boxes are filtered by area threshold and mapped to normalized YOLO format: [x_center, y_center, w, h] in [0, 1].

C. ModelTrainer Component (model_trainer.py)

Class: ModelTrainer(config: ModelTrainerConfig)

Method: initiate_model_trainer(data_yaml: Path, output_dir: Path) -> tuple[YOLO, Path]

Hyperparameters: Epochs=50, Image Size=512, Batch=16, Optimizer=AdamW, Cosine LR (lr0=0.01, lrf=0.01), mosaic=1.0.

D. PredictPipeline — ONNX Runtime Auto-Select (prediction_pipeline.py)

Class: PredictPipeline(model_path=None, conf=0.25, iou=0.45)

Attributes: engine_type: str — 'ONNX Runtime' or 'PyTorch'

Auto-Select Logic: At initialization, checks for best.onnx in project root. If present, loads via YOLO('best.onnx') using ONNX Runtime backend (CPUExecutionProvider with SIMD + graph fusion). Falls back to best.pt PyTorch if ONNX not found. Sets engine_type accordingly.

Method: predict(image_path, conf, iou, _timings: dict) -> list[list[float]] — delegates to ModelMonitoring.

E. ModelMonitoring — Smart Tile Early-Exit + Timing (model_monitoring.py)

Method: predict_large_image(image_path, model_path, tile_size=512, overlap=100, conf_threshold=0.25, iou_threshold=0.45, batch_size=16, _timings=None)

Background Tile Rejection Algorithm (per chip):

1. Subsample chip at 4px stride: sample = chip[:, :4, :4]
2. Compute mean (mu) and std (sigma) of subsampled pixel values
3. Reject if: sigma < 5.0 (ocean/uniform terrain)
4. Reject if: mu < 4.0 AND sigma < 3.0 (void/border padding)
5. Reject if: mu > 245.0 AND sigma < 4.0 (cloud whiteout)

Multi-Phase Timing Instrumentation: Records t_load_ms (raster read + normalize), t_inference_ms (batched forward passes), t_nms_ms (batched NMS), tiles_processed, tiles_skipped into _timings dict.

Coordinate Mapping: $X_{global} = u_{local} + x_{offset}$, $Y_{global} = v_{local} + y_{offset}$. Deduplication via torchvision.ops.batched_nms at tau_iou = 0.45.

2. High-Threat Intelligence Brief Generation

The Flask application generates a natural-language threat intelligence brief from each inference result. Detections with confidence ≥ 0.75 (HIGH_CONF_THRESHOLD) are grouped by class name and counted. Three output cases are handled:

Case 1 — No High-Confidence Detections: Emits a notice recommending lowering the confidence threshold for a broader scan.

Case 2 — Single High-Confidence Class: Emits a targeted statement naming the class, count, and 'primary threat' designation.

Case 3 — Multiple High-Confidence Classes: Emits a multi-class brief listing the top 3 classes with counts (plus remainder count), total threat count, and class count. Appends 'warrant priority review' to flag for analyst attention.

Template variables passed: threat_brief (string), high_threat_objects (list of {name, count}), high_threat_total (int).

3. Per-Phase Latency Tracker Schema

The latency_tracker dict is constructed in application.py and passed to the template for UI rendering:

Field	Type	Description
total_str	str	Human-readable total inference time (e.g. '1.24 s' or '845 ms')
total_ms	float	Total inference time in milliseconds
tiles_processed	int	Number of tiles that passed background rejection and entered the network
tiles_skipped	int	Number of tiles rejected by early-exit (ocean, padding, clouds)
engine_type	str	'ONNX Runtime' or 'PyTorch' — displayed in engine badge
phases[]	list[dict]	4 entries: {name, time_str, pct, color} for Raster Load, Forward Pass, NMS, Overlay Render

4. Mathematical Formulations & Optimization Loss

A. Total Loss Objective Function

The network optimizes a multi-task composite loss balancing localization, classification, and boundary regression:

$$L_{total} = \lambda_{box} L_{CloU} + \lambda_{cls} L_{BCE} + \lambda_{dfl} L_{DFL}$$

B. Complete Intersection over Union (CloU Loss)

CloU incorporates overlap area, central point distance, and aspect ratio consistency:

$$L_{CloU} = 1 - IoU + (\rho^2(\mathbf{b}, \mathbf{b}^{gt}) / c^2) + \alpha v$$

where v measures aspect ratio discrepancy: $v = (4 / \pi^2)(\arctan(w^{gt}/h^{gt}) - \arctan(w/h))^2$

C. Global Coordinate Mapping & Batched NMS

For chip offset (x_{off} , y_{off}), predicted local coordinates [u_1 , v_1 , u_2 , v_2] are mapped to global pixel coordinates:

$$x_1 = u_1 + x_{off}, \quad y_1 = v_1 + y_{off}, \quad x_2 = u_2 + x_{off}, \quad y_2 = v_2 + y_{off}$$

Cross-tile deduplication via torchvision.ops.batched_nms at threshold $\tau_{iou} = 0.45$.

5. API Specifications & Data Contracts

Endpoint	Method	Request Payload	Response Structure
/	GET	None	HTML upload form with confidence/IoU sliders
/	POST	multipart/form-data - file: GeoTIFF (.tif) - conf: float [0..1] - iou: float [0..1]	Rendered template with 8 result panels: stats bar, engine badge, threat brief, latency tracker, metadata, detection overlay, class breakdown, detection log.
/health	GET	None	{ "status": "ok", "service": "satellite-threat-detection", "deployment": "active" }

6. Telemetry Log Record Schemas

Detection Record Structure (log_detection_details):

Each detection event in logs/detection/detection_*.log contains: (1) File metadata (dimensions, bands, byte size), (2) Hyperparameter state (conf, iou), (3) Engine type and per-phase latency breakdown (t_load_ms, t_inference_ms, t_nms_ms, t_render_ms, tiles_processed, tiles_skipped), (4) High-threat detection count (conf >= 0.75), (5) Class breakdown counts & percentages, and (6) Itemized detection table: Detection Index, Class Name, Class ID, Confidence %, and Global Bounding Box [x1, y1, x2, y2].